

Convolutional Neural Network

Homework 3
COSC 4337
Dr. Rizk

About

In this homework we will create a Convolutional Neural Network (CNN) from scratch in Python. We will implement things such as a Reshape Layer, the Binary Cross Entropy Loss, and the Sigmoid Activation, etc. Finally, we'll use all these objects to make a neural network capable of classifying handwritten digits from the MNIST dataset. Before beginning, make sure you're comfortable calculating cross correlation and convolution from a given input and kernel. See the next 2 slides.

Be sure to download the starter code files. You're welcome to transfer the code to a jupyter notebook if you'd rather code in there.

Note: Don't worry if your code doesn't work 100% or if you get poor results when training. What matters most is your effort and understanding, so leave comments documenting things where possible and the TA will grade you based on your understanding of CNN and if you're on the right track.

Cross-Correlation

Given the following input and kernel, what is the cross correlation (output)?

1	6	2
5	3	1
7	0	4

input

 $\star$

1	2
-1	0

kernel

 $=$

output

Learn more about cross-correlation here:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.correlate.html>

Convolution

Using the given input and kernel, what is the convolution (output)? Hint: Rotate the kernel 180.

1	6	2
5	3	1
7	0	4

input

 $\star$

1	2
-1	0

kernel

 $=$

output

The Convolutional Class

We'll start by creating our convolutional class, which is where we'll implement a forward and backward pass. Here's the following that we'll start with. See more details in next slide.

```
1  import numpy as np
2  from scipy import signal
3  from layer import Layer
4
5  class Convolutional(Layer):
6      def __init__(self, input_shape, kernel_size, depth):
7          input_depth, input_height, input_width = input_shape
8          self.depth = depth
9          self.input_shape = input_shape
10         self.input_depth = input_depth
11         self.output_shape = (depth, input_height - kernel_size + 1, input_width - kernel_size + 1)
12         self.kernels_shape = (depth, input_depth, kernel_size, kernel_size)
13         self.kernels = np.random.randn(*self.kernels_shape)
14         self.biases = np.random.randn(*self.output_shape)
```

The Convolutional Class

- The constructor takes as input the `input_shape` (a tuple containing the depth, height, and width of the input), the `kernel_size` (a number representing the size of each matrix inside each kernel), and the `depth` (a number representing how many kernels we want) aka the depth of the output.
- We've computed the output and kernel shape for you, but be sure that you understand why those are the correct shapes.
- Finally, the kernels and biases are initialized randomly.

Implementing Forward

- Your job is to implement the forward method, which takes the input and should compute the following formula. Y is the output, B is the bias matrix, X is the input, k is the kernel. Note that the star notation stands for the cross correlation that you computed earlier.

$$Y_i = B_i + \sum_{j=1}^n X_j \star K_{ij}, \quad i = 1 \dots d$$

Implementing Forward

You should have the following started code in the convolutional.py file. Please implement the forward method here or in a jupyter notebook if you transferred the code over.

```
def forward(self, input):  
    self.input = input  
    self.output = None  
    # TODO: Implement the forward method using the formula provided in the powerpoint.  
    # You may add or remove any variables that you wish.  
    return self.output
```


Implementing Backward

Your next task is to implement the backward method. Note, you need to compute 3 things here. The derivative of the error (E) with respect to the trainable parameters (the kernel and biases), and the derivative of E with respect to the input of the layer. You may find the following equations helpful.

$$\frac{\partial E}{\partial K_{ij}} = X_j \star \frac{\partial E}{\partial Y_i}$$

$$\frac{\partial E}{\partial B_i} = \frac{\partial E}{\partial Y_i}$$

$$\frac{\partial E}{\partial X_j} = \sum_{i=1}^n \frac{\partial E}{\partial Y_i} \overset{*}{f}_{ull} K_{ij}$$

Note that *full stands for the full convolution

Implementing Backward

You should have the following started code in the convolutional.py file. Please implement the backward method here or in a jupyter notebook if you transferred the code over.

```
def backward(self, output_gradient, learning_rate):  
    # TODO: initialize the kernels_gradient and input_gradient.  
    kernels_gradient = None  
    input_gradient = None  
  
    # TODO: implement the back pass here. The equations in the ppt may help, but you're free to  
    # add as much or as little code as you'd like.  
  
    # TODO: update the kernels and biases  
    self.kernels -= None  
    self.biases -= None  
  
    return input_gradient
```

Implementing Reshape

The output of the convolutional layer is a 3d block, so we'll need to reshape this into a column vector before passing into our final dense layer for classification.

You can find the starter code in `reshape.py`, or you can transfer the next slide into your notebook.

Implementing Reshape

```
1  import numpy as np
2  from layer import Layer
3
4  class Reshape(Layer):
5      def __init__(self, input_shape, output_shape):
6          self.input_shape = input_shape
7          self.output_shape = output_shape
8
9      def forward(self, input):
10         # TODO: reshape the input to the output_shape and return it.
11         pass
12
13     def backward(self, output_gradient, learning_rate):
14         # TODO: reshape the output to the input_shape and return it.
15         pass
```

Implementing Dense Layer

Let's implement our dense layer. The started code is located in dense.py This should be similar to previous homeworks when you learned about neural networks.

- The constructor takes the input_size and output_size. Implement the forward method. Recall formula for a dense layer is just: $y = xA^T + b$. You can read more about linear / dense layers here.
<https://pytorch.org/docs/stable/generated/torch.nn.Linear.html>
- Implement the backward method.

Implementing Dense Layer

```
import numpy as np
from layer import Layer

class Dense(Layer):
    def __init__(self, input_size, output_size):
        self.weights = np.random.randn(output_size, input_size)
        self.bias = np.random.randn(output_size, 1)

    def forward(self, input):
        # TODO: apply linear transformation to the input. see ppt for equation.
        pass

    def backward(self, output_gradient, learning_rate):
        # TODO: update the weights and bias
        pass
```

Implementing Binary Cross Entropy Loss

Why is binary cross entropy loss a good choice here?

Implementing Binary Cross Entropy Loss

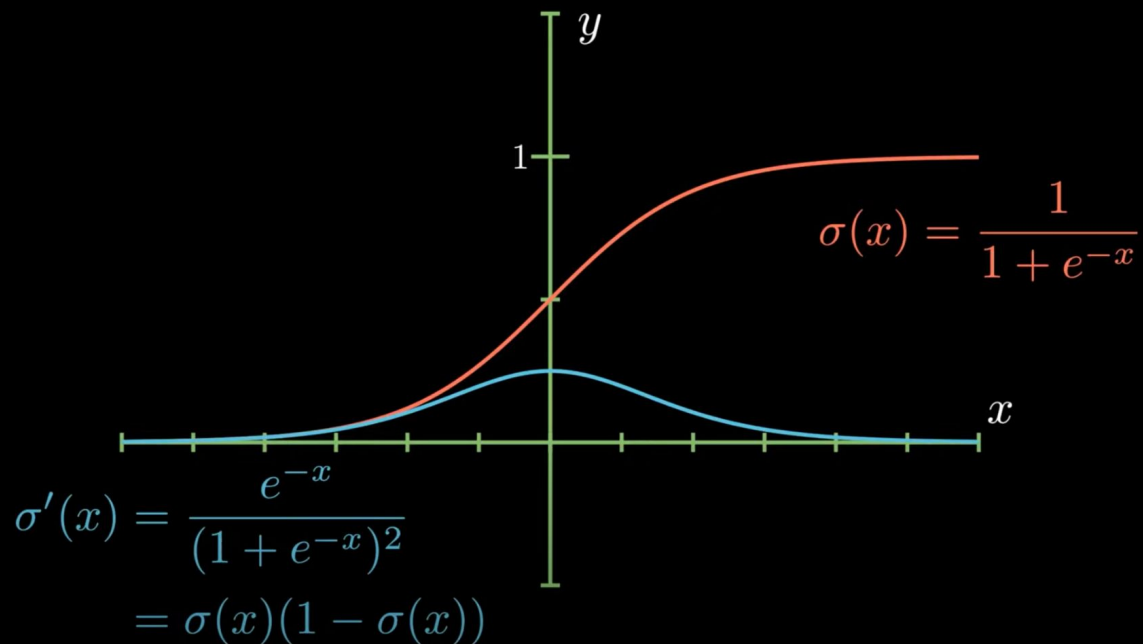
Given the desired output (y^*) and the network's output (y), we will need to calculate the following. You can find the starter code in `losses.py`

$$E = -\frac{1}{n} \sum_{i=1}^n y_i^* \log(y_i) + (1 - y_i^*) \log(1 - y_i)$$

$$\frac{\partial E}{\partial y_i} = \frac{1}{n} \left(\frac{1 - y_i^*}{1 - y_i} - \frac{y_i^*}{y_i} \right)$$

Implementing Sigmoid Activation

Recall that the sigmoid is defined as



Implementing Sigmoid Activation

You can find the starter code in activations.py

```
class Sigmoid(Activation):  
    def __init__(self):  
        def sigmoid(x):  
            # TODO: return the sigmoid of x  
            pass  
  
        def sigmoid_prime(x):  
            # TODO: return the derivative  
            pass  
  
        super().__init__(sigmoid, sigmoid_prime)
```

Putting it all together

We now have all of the pieces to put our network together and begin training. We've provided most of the code for you in `mnist_conv.py`, but would like for you to implement the following:

- Create a list (network) containing all of our layers. It should look something like Convolution → Sigmoid → Reshape → Dense → Sigmoid. Note that the image sizes are $28 * 28$ and you may experiment with kernel sizes of your choice. Try to keep things small to avoid long wait times.
- Fill in / finish the main training loop. Note, this should look standard to most neural network training loops.

Putting it all together

```
# TODO: Add our layers and the flow of input into this list.  
network = []
```

```
# train  
for e in range(epochs):  
    error = 0  
    for x, y in zip(x_train, y_train):  
        # forward  
        output = x  
        for layer in network:  
            output = layer.forward(output)  
  
        # TODO: update our error  
        error += None  
  
        # TODO: perform back prop  
  
    error /= len(x_train)  
    print(f"{e + 1}/{epochs}, error={error}")  
  
# TODO: run the test data through and print out your predictions  
for x, y in zip(x_test, y_test):  
    pass
```

Congratulations! 😊

Making these networks from scratch is a great way to learn in detail what's going on. Feel free to play around with this or keep implementing other models / algorithms from scratch as practice. Please reach out to the TA or revisit lecture slides if you're still iffy on CNN.